

Phase 2 Design Document

Overall System Design

Bazar.com is implemented as a distributed microservice bookstore system. The client sends all requests to the frontend service, which acts as the public API gateway. The frontend does not permanently store books or orders. Instead, it forwards requests to backend replicas and manages caching and load balancing.

The system contains five Docker services:

Service	Port	Responsibility
Frontend	3000	Public API, load balancing, and caching
Catalog Replica 1	3001	Stores and updates book information
Catalog Replica 2	3003	Replica of catalog1
Order Replica 1	3002	Handles purchases and order storage
Order Replica 2	3004	Replica of order1

All services communicate using REST APIs through Docker Compose networking.

The catalog services store book information inside CSV files:

```
catalog1.csv
catalog2.csv
```

The order services store purchased orders inside:

```
orders1.csv
orders2.csv
```

System Workflow

Search Request

When a client sends:

```
GET /search/:topic
```

the frontend checks whether the search result already exists in cache. If the result is not cached, the frontend forwards the request to one of the catalog replicas using round-robin load balancing. The catalog service reads the CSV file and returns matching books. The frontend stores successful results in memory cache for future requests.

Info Request

When the client sends:

```
GET /info/:id
```

the frontend first checks its in-memory cache.

If the book data already exists, the frontend immediately returns the cached result and logs a cache hit. If the data does not exist, the frontend forwards the request to one of the catalog replicas and stores the response in cache.

This reduces repeated requests to the catalog services and improves performance.

Purchase Request

When the client sends:

```
POST /purchase/:id
```

the frontend forwards the request to one of the order replicas using round-robin load balancing.

The order service:

1. Requests book information from the catalog service.
2. Checks whether quantity is greater than zero.
3. Invalidates stale frontend cache using:

```
DELETE /cache/:id
```

4. Updates the catalog quantity.
5. Saves the order into the local orders CSV file.
6. Synchronizes the order with the other order replica.
7. Returns a success response to the frontend.

The catalog replica also synchronizes updates with the other catalog replica to keep quantities consistent.

Load Balancing

The frontend uses simple round-robin load balancing.

Catalog requests alternate between:

```
catalog1 → catalog2 → catalog1
```

Order requests alternate between:

```
order1 → order2 → order1
```

This distributes requests evenly between replicas and simulates a distributed environment.

Cache and Consistency

The frontend uses a simple in-memory JavaScript object for caching.

Cached requests:

- `GET /info/:id`
- `GET /search/:topic`

Write operations such as purchases are never cached.

When a purchase occurs, the frontend cache entry for that book is removed to prevent stale data. The next request becomes a cache miss and retrieves fresh information from the catalog replica.

Replica synchronization is implemented using internal synchronization APIs between replicas. Catalog replicas synchronize quantity updates, and order replicas synchronize new order records.

Design Tradeoffs

CSV files were used instead of databases to simplify implementation and make the system easier to run and test. However, CSV storage is not ideal for concurrent writes or large-scale applications.

The frontend cache is lightweight and fast, but cached data is lost if the frontend container restarts.

Round-robin load balancing is simple and easy to implement, but it does not check replica health or current server load.

Possible Improvements

Possible future improvements include:

- Replacing CSV files with MySQL or PostgreSQL
 - Adding user authentication
 - Using Redis for distributed caching
 - Adding automatic failover when replicas become unavailable
 - Adding stronger synchronization and retry mechanisms
 - Implementing advanced load balancing algorithms
-

How To Run The Program

1. Open Docker Desktop.
2. Open the project folder in the terminal.
3. Run:

```
docker compose up --build -d
```

4. Test the APIs:

Search Request

```
http://localhost:3000/search/distributedsystems
```

Info Request

```
http://localhost:3000/info/2
```

Purchase Request

```
curl -X POST http://localhost:3000/purchase/2
```

5. Stop the system:

```
docker compose down
```